

Real-Time Multi-Camera Intelligent Vision System for Smart Factory Safety

Technologies: NVIDIA DeepStream, Transfer Learning Toolkit (TLT), TensorRT, NVIDIA Jetson
Candidate: Sai Kumar

1. Objective

The objective of this work is to design a real-time multi-camera computer vision system deployable on edge devices under practical constraints such as limited compute, memory, power, latency, and reliability. The system targets industrial smart factory environments, where safety-critical decisions must be executed locally with minimal dependence on cloud connectivity. The design emphasizes engineering judgment, scalability, and production feasibility using NVIDIA's edge AI stack.

2. Problem Statement & Use Case

Use Case: Smart Factory Safety & Operations Monitoring

Industrial environments require continuous monitoring to ensure worker safety, enforce restricted zones, and analyze operational efficiency. Manual supervision is error-prone and does not scale with facility size.

Problem Statement

Design a real-time multi-camera intelligent vision system deployed on edge devices to monitor a smart factory floor for worker safety, operational efficiency, and anomaly detection. The system must support object detection, semantic segmentation, video analytics, image stitching, and pattern discovery under strict constraints of latency, power, and compute, using NVIDIA DeepStream, TLT, and TensorRT.

3. Hardware & Platform Constraints

Target Platform

- NVIDIA Jetson Xavier NX / Orin Nano

- Optional auxiliary nodes (e.g., Raspberry Pi) for logging and monitoring

Constraints

Resource	Constraint
Power	15–25 W
Memory	~8 GB
Cameras	4–8 RTSP streams
Latency	< 100 ms per stream
Thermal	Limited / fanless

Jetson platforms are selected for their native CUDA, TensorRT, and DeepStream support, enabling efficient multi-stream inference. Cloud-based inference is avoided due to latency, privacy, and network reliability concerns common in industrial deployments.

4. Architecture Overview

Camera Layer

- Fixed RTSP IP cameras with overlapping fields of view
- 1080p input, downscaled to 720p for inference

Edge Layer (Jetson)

- DeepStream-based multi-stream pipeline
- TensorRT-optimized inference
- Real-time alerts and short-term local storage

Cloud Layer

- Long-term metadata storage
- Dashboards and reporting
- Model retraining and OTA updates

Latency-sensitive tasks are executed at the edge, while compute-intensive and non-real-time workloads are handled in the cloud.

5. DeepStream Inference Pipeline

Pipeline Design

RTSP Cameras

→ nvv4l2decoder

→ nvstreammux

→ nvinfer (Object Detection)

→ nvtracker

→ nvdsanalytics

→ Secondary Inference (Segmentation – low FPS)

→ Sink (Display / Storage / Message Broker)

Design Rationale

- **nvv4l2decoder:** Hardware-accelerated decode to reduce CPU load
- **nvstreammux:** Batches multiple streams for efficient GPU utilization
- **nvinfer:** Executes TensorRT engines for low-latency inference
- **nvtracker:** Assigns persistent IDs for temporal analytics
- **nvdsanalytics:** Enables counting, zone violations, and loitering detection

Object detection and tracking run on every frame, while segmentation and stitching are executed periodically to conserve compute and power.

6. Model & AI Pipeline Design

Task-to-Model Mapping

Task	Model	Rationale
Object Detection	YOLOv8-Nano / EfficientDet-D0	High FPS, small footprint
Semantic Segmentation	DeepLabV3 + MobileNet	Lightweight backbone
Tracking	NvDCF	GPU-accelerated
Anomaly Detection	Autoencoder / Isolation Forest	Unsupervised, low compute
Clustering	DBSCAN / K-Means	Noise-tolerant

Key Design Decisions

- Transfer Learning Toolkit (TLT) is used to fine-tune pretrained detection models with limited factory-specific data.
- Semantic segmentation provides contextual zone awareness and is not executed per frame.
- Unsupervised learning is used for anomaly detection due to the scarcity of labeled abnormal events.
- Pattern discovery is performed on trajectory and dwell-time features to identify congestion and unsafe behavior trends.

7. Optimization & Deployment

Optimization Strategy

- FP16 precision for segmentation models
- INT8 precision for object detection using calibration datasets
- TensorRT optimizations including layer fusion, kernel auto-tuning, and engine caching

Minor accuracy degradation is accepted in favor of deterministic real-time performance.

Deployment

- Docker-based containerization for reproducibility and OTA updates
 - Separate services for inference, analytics, and messaging
 - Continuous monitoring of FPS, latency, GPU utilization, and memory usage
-

8. Trade-Off Analysis

- **Accuracy vs Latency:** Lightweight models prioritized to meet real-time constraints
- **Model Size vs Performance:** Mobile and Nano backbones selected over larger architectures
- **Edge vs Cloud:** Edge ensures low latency and reliability; cloud provides scalability
- **Image Stitching:** Periodic execution instead of continuous processing to save resources

Design choices intentionally favor predictable performance over peak accuracy.

9. Failure Scenarios & Recovery

- **Network Failure:** Edge continues inference; metadata cached and synced later
 - **Hardware/Software Crash:** Docker auto-restart with watchdog monitoring
 - **Thermal Throttling:** Adaptive load shedding (reduced segmentation frequency)
 - **Model Drift:** Confidence monitoring, periodic retraining, and OTA model updates
-

10. Assumptions

- Fixed camera placement
 - Moderate lighting variation
 - 4–8 concurrent video streams
 - Internet connectivity not guaranteed
 - Industrial privacy requirements apply
-

11. Conclusion

This design demonstrates a production-ready edge AI vision system for smart factory environments. By leveraging NVIDIA DeepStream, TLT, and TensorRT, the system balances real-time performance, scalability, and reliability under strict edge constraints, making it suitable for safety-critical industrial deployments.